

Generic Recommender System

Mateusz Bernart
156072

Patryk Janiak
156053

Marek Seget
156042

January 10, 2026

1 Introduction

Recommender systems are critical components for personalizing user experiences. This report details the development of a generic recommender system designed to handle diverse data sources through a unified graph-based representation.

The primary challenge in building a generic system is the variability of dataset formats. To address this, we implement a "Convert-First" strategy. Raw data is first transformed into a standardized JSON structure, unifying user IDs, item IDs, and interaction timestamps. This unified format is then ingested by a generic interface to construct a heterogeneous graph connecting users, items, and feature nodes. This structure effectively captures complex relationships and serves as a robust foundation for training various recommendation algorithms, ranging from traditional Matrix Factorization to advanced Graph Neural Networks (GNNs).

2 Related Work

Recent advancements in recommender systems have increasingly leveraged graph-based methods to capture high-order connectivity between users and items.

Schlichtkrull et al. introduced Relational Graph Convolutional Networks (R-GCNs), demonstrating the efficacy of modeling relational data for link prediction tasks [3]. This foundational work supports our use of heterogeneous graphs to model diverse interactions (e.g., "watched", "owned", "rated").

Further exploring inductive capabilities, Teru et al. proposed subgraph reasoning methods for relation prediction, allowing models to generalize to unseen nodes—a critical feature for handling cold-start scenarios in dynamic systems [2].

More recently, Zhang and Liu have explored path-based neural networks for inductive link prediction, emphasizing the importance of preserving local graph topology when generating embeddings for new entities [1].

3 Dataset Description

To validate the generic nature of our framework, we merged two distinct datasets into a single homogeneous structure.

3.1 MovieLens

The preprocessed MovieLens dataset represents the domain of explicit feedback.

- **Nodes:** 162,342 Users, 62,423 Movies, 20 Genres.
- **Edges:** 25,000,095 Explicit Ratings (normalized to 0.5-5.0) and 12,452,811 Implicit interactions (filtered for ratings ≥ 4.0).

3.2 Steam Reviews

The Steam dataset represents the gaming domain, characterized by implicit feedback (ownership) and rich metadata.

- **Nodes:** 22,077 Users, 32,133 Games, 22 Genres.
- **Edges:** 45,261 User-Game interactions and 72,786 Game-Genre associations.

3.3 Combined Heterogeneous Graph

The final merged graph allows for simultaneous training across domains:

- **Total Users:** 184,419
- **Total Items:** 43,660 (Movies + Games)
- **Total Interactions:** 12,497,401 User-Item edges.

4 Method Description

4.1 System Architecture

The system workflow follows a three-stage pipeline designed to handle data heterogeneity:

1. **Data Ingestion & Preprocessing:** Raw data is ingested from diverse sources (CSVs, JSONs). We apply a "Convert-First" strategy, transforming all input data into a standardized JSON structure that unifies User IDs, Item IDs, and interaction timestamps. Explicit ratings are normalized to a 0.5-5.0 scale, while implicit feedback (e.g., game ownership) is processed as binary interactions.
2. **Graph Construction:** The standardized data is merged into a single heterogeneous graph. This structure connects *User*, *Item*, and *Feature* (e.g., Genre) nodes, allowing for the modeling of complex, multi-hop relationships.
3. **Model Training & Execution:** The graph serves as the input for embedding generation. We employ a modular interface where models inherit from a common `EmbeddingGenerator` class, ensuring consistent training and inference APIs.

Please refer to **Appendix A** (Figure 3) for the complete workflow diagram.

4.2 Algorithms

We evaluated five distinct model strategies, ranging from traditional factorization to state-of-the-art graph neural networks:

- **Matrix Factorization (MF):** A traditional collaborative filtering method that decomposes the user-item interaction matrix into lower-dimensional latent vectors. Ratings are predicted via the dot product of these vectors. While computationally efficient, it is limited to capturing linear relationships in explicit feedback data.
- **Neural Collaborative Filtering (NCF):** A deep learning evolution of MF that models complex user-item relationships. It employs a dual-pathway architecture:
 - *Generalized Matrix Factorization (GMF):* Captures linear interactions.
 - *Multi-Layer Perceptron (MLP):* Captures non-linear patterns that simple dot products miss.
- **LightGCN:** A simplified Graph Convolutional Network (GCN) specifically optimized for recommendation. Unlike standard GCNs, LightGCN removes heavy feature transformations and non-linear activation functions. It propagates embeddings linearly across the user-item graph, efficiently capturing high-order connectivity (e.g., "friends of friends") directly from the graph structure.

- **Node2Vec:** A feature learning algorithm based on biased Random Walks. It generates embeddings by simulating walks across the heterogeneous graph (Users, Items, Genres). This approach preserves both the *local neighborhood* (homophily) and *structural roles* (equivalence) of nodes, balancing community detection with global structural analysis.
- **Cleora:** An ultra-fast, memory-efficient graph embedding algorithm designed for massive scalability. It operates without an Artificial Neural Network (ANN), instead iteratively refining embeddings based on local neighborhood data via graph walks. This makes it particularly suitable for extremely large datasets where other models face memory constraints.

5 Evaluation Strategy

5.1 Inference Pipeline

To evaluate the models in a realistic setting, we implemented a complete inference pipeline that simulates a production recommender system:

1. **Graph Retrieval:** The system validates the input User ID against the heterogeneous graph.
2. **Cold Start Handling:** If the user does not exist in the graph, the system returns a fallback list of globally "Popular Items".
3. **Candidate Scoring:** For existing users, the system fetches the pre-computed user embedding and interaction history. It then computes similarity scores (e.g., cosine similarity) between the user vector and all candidate item vectors.
4. **Filtering & Ranking:** Items already present in the user's interaction history are filtered out. The remaining candidates are sorted by prediction score to produce the final Top-N recommendation list.

Please refer to **Appendix A** (Figure 4) for the complete inference diagram.

5.2 Evaluation Metrics

We assessed performance using a suite of information retrieval metrics at $K \in \{20, 50, 100, 1000\}$ to measure top-list quality:

- **Precision @ K:** The ratio of relevant items in the top-K recommendations.
- **HitRate @ K:** The percentage of users for whom at least one relevant item appears in the top-K.
- **NDCG @ K (Normalized Discounted Cumulative Gain):** Measures the quality of ranking, giving higher importance to relevant items appearing earlier in the list.
- **MAP @ K (Mean Average Precision):** The average precision for each user, averaged over all users.

Beyond accuracy, we analyzed the "**Discovery**" capabilities of each model using metrics for **Diversity** (variety of item types), **Novelty** (ability to recommend less popular items), and **Serendipity**.

6 Outcomes and Conclusions

6.1 Performance Metrics: Accuracy & Relevance

We evaluated the models on four key metrics at $K = 20$. As shown in Table 1, graph-based models (Node2Vec, LightGCN) significantly outperform traditional methods in relevance tasks.

Table 1: Model Performance Comparison @ K=20.

Model	Precision	HitRate	NDCG	MAP
LightGCN	0.0030	0.0595	0.0216	0.0114
Node2Vec	0.0030	0.0603	0.0220	0.0117
Matrix Factorization	0.0012	0.0238	0.0092	0.0052
NCF	0.0004	0.0083	0.0030	0.0016
Cleora	0.0001	0.0014	0.0005	0.0002

The results indicate that Node2Vec achieves the highest HitRate (6.03%) and Ranking quality (NDCG 0.0220), closely followed by LightGCN. In contrast, Cleora and NCF struggle to predict "ground truth" interactions in this sparse, heterogeneous setting.

6.2 Novelty & Exploration

While accuracy metrics favor graph neural networks, the analysis of "Discovery" metrics reveals a drastic trade-off. We measured models on **Diversity** (variety of content), **Novelty** (unpopularity of recommendations), and **Serendipity** (unexpected relevance).

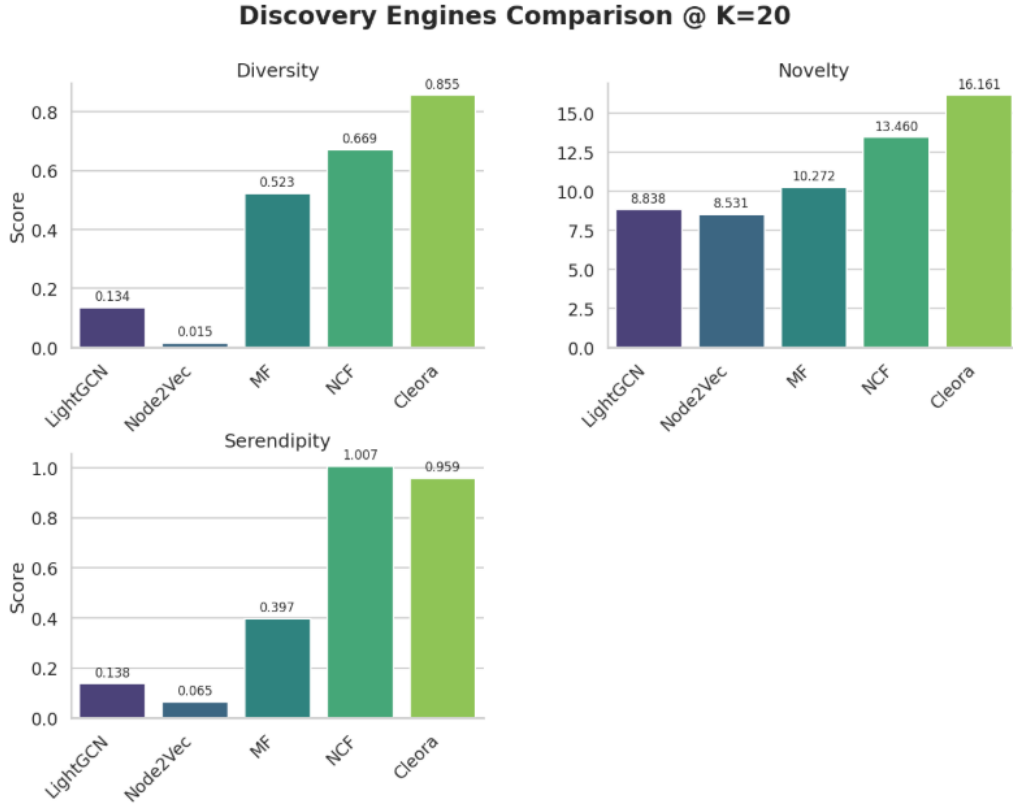


Figure 1: Cleora and NCF significantly outperform accuracy-focused models in Novelty and Serendipity, highlighting their utility for exploration tasks.

- **Novelty Leaders:** Cleora and NCF act as "Discovery Engines." Cleora achieves the highest Novelty score (**16.161**), recommending items with an average popularity of just 28 interactions, whereas Node2Vec tends to recommend popular items (average popularity 1150).
- **Diversity:** Cleora also leads in Diversity (**0.855**), followed by NCF (0.669). The accurate models, Node2Vec (0.015) and LightGCN (0.134), suffer from popularity bias, resulting in very low diversity scores.

- **Serendipity:** NCF achieved the highest Serendipity score (**1.007**), suggesting it is most effective at finding relevant items that are also surprising to the user.

6.3 The Accuracy-Diversity Trade-off

The experimental results highlight a clear distinction between "Safe" and "Discovery" engines:

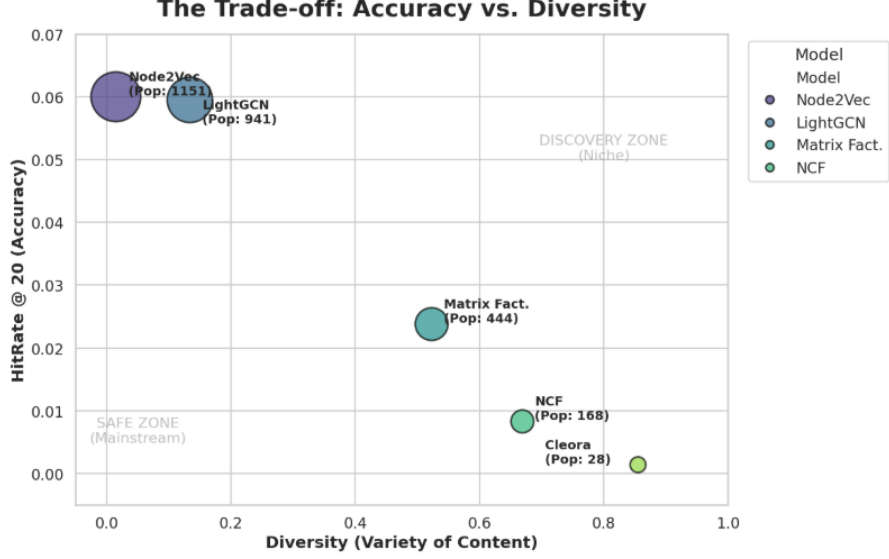


Figure 2: LightGCN and Node2Vec occupy the 'Safe Zone', while Cleora and NCF inhabit the 'Discovery Zone'.

- **The Safe Zone:** Dominant in relevance but low in exploration. Ideal for primary feeds where trust is essential.
- **The Discovery Zone:** Excellent for surfacing the "long tail" of content but less reliable for immediate relevance.

6.4 Final Recommendations

Based on this analysis, we propose a hybrid deployment strategy:

1. **Primary Feed:** Use **LightGCN** for high accuracy and user satisfaction.
2. **Discovery Section:** Use **Matrix Factorization** or **NCF** to balance relevance with diversity ("Because You Watched").
3. **Deep Dive:** Use **Cleora** to surface niche content that users rarely discover through popularity-biased models.

References

- [1] Canlin Zhang and Xiuwen Liu. Inductive Link Prediction in Knowledge Graphs using Path-based Neural Networks. *arXiv preprint arXiv:2312.10293*, 2023.
- [2] Komal K. Teru, Etienne Denis, and William L. Hamilton. Inductive Relation Prediction by Subgraph Reasoning. *arXiv preprint arXiv:1911.06962*, 2020.
- [3] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling Relational Data with Graph Convolutional Networks. *arXiv preprint arXiv:1703.06103*, 2017.

A Appendix

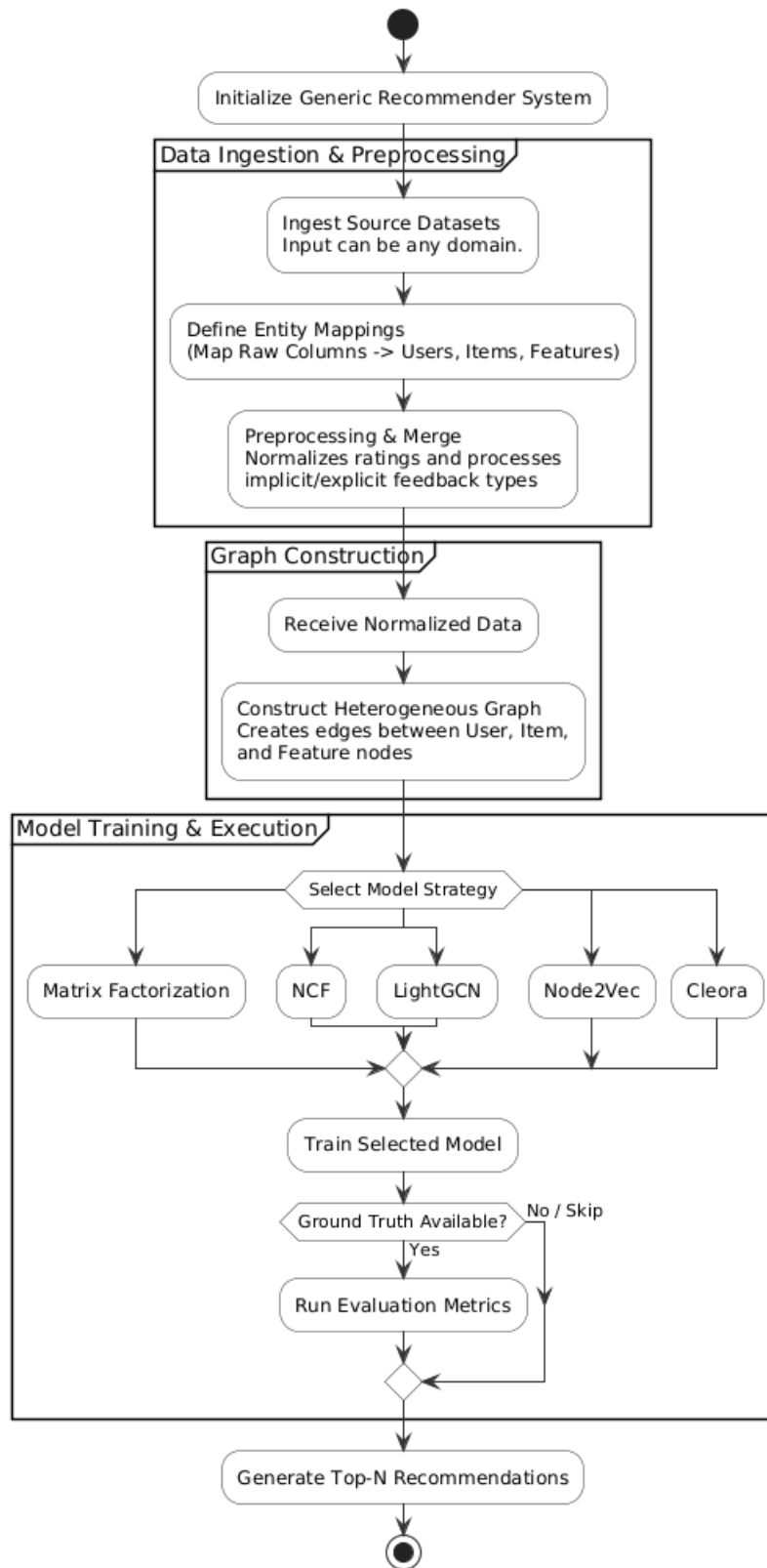


Figure 3: Generic Recommender System Workflow: From Data Ingestion to Recommendation Generation

